

-  **OmnID: Sovereign, Post-Quantum, and Decentralized Identity System**
 -  **Arquitectura del Sistema**
 -  **Especificaciones Técnicas y Criptográficas**
 -  **Estructura del Repositorio**
 -  **Componentes Implementados**
 - 1. Interfaz de Simulación (Frontend)
 - 2. API Criptográfica (Backend)
 - 3. Cumplimiento GDPR (Backend Securizado)
 -  **Guía de Instalación y Ejecución**
 - Requisitos Previos
 - Paso 1: Configurar el Servidor
 - Paso 2: Ejecutar los Tests de Integración
 - Paso 3: Levantar la API
 - Paso 4: Ejecutar el Frontend
 -  **Referencia de la API (Endpoints)**
 - 1. Estado de la Red
 - 2. Emisión de Cédulas Digitales (Enrolamiento)
 - 3. Verificación de Identidad (Auditoría ZKP)
 - 4. Endpoints de Cumplimiento GDPR
 -  **Plan de Continuidad y Próximas Fases**
 -  **Licencia**

OmnID: Sovereign, Post-Quantum, and Decentralized Identity System

License **AGPL v3** node **>=20.0.0** NIST PQ **FIPS 203 / 204** W3C DID **v1.0**

OmnID es un ecosistema técnico de Identidad Digital Soberana (SSI) de grado gubernamental diseñado para ser inmune a la falsificación, la suplantación de identidad y futuros ataques mediante computadoras cuánticas. El sistema integra Tecnología de Registro Distribuido (DLT), criptografía basada en redes de celosías (Lattice-based Cryptography), y mecanismos de divulgación selectiva para garantizar el control total del usuario sobre sus datos.



Arquitectura del Sistema

OmniiD se estructura bajo el modelo clásico de **Identidad Soberana (SSI)**, compuesto por tres roles fundamentales: **Emisor (Issuer)**, **Propietario (Holder)**, y **Verificador (Verifier)**, soportado sobre una red descentralizada de confianza.

```
Parse error on line 7:
...      subgraph Emisor (Gobierno/Entidad de
-----^
Expecting 'SEMI', 'NEWLINE', 'SPACE', 'EOF', 'GRAPH', 'DIR',
'TAGEND', 'TAGSTART', 'UP', 'DOWN', 'subgraph', 'end', 'SQE',
'PE', '-)', 'DIAMOND_STOP', 'MINUS', '--', 'ARROW_POINT',
'ARROW_CIRCLE', 'ARROW_CROSS', 'ARROW_OPEN',
'DOTTED_ARROW_POINT', 'DOTTED_ARROW_CIRCLE',
'DOTTED_ARROW_CROSS', 'DOTTED_ARROW_OPEN', '==',
'THICK_ARROW_POINT', 'THICK_ARROW_CIRCLE', 'THICK_ARROW_CROSS',
'THICK_ARROW_OPEN', 'PIPE', 'STYLE', 'LINKSTYLE', 'CLASSDEF',
'CLASS', 'CLICK', 'DEFAULT', 'NUM', 'PCT', 'COMMA', 'ALPHA',
'COLON', 'BRKT', 'DOT', 'PUNCTUATION', 'UNICODE_TEXT', 'PLUS',
'EQUALS', 'MULT', got 'PS'
```

Especificaciones Técnicas y Criptográficas

Módulo	Estándar / Algoritmo	Implementación en OmniID	Cumplimiento
Identificadores	W3C DID	did:omni:<Unique-ID>	W3C DID v1.0
Cifrado FPE	FF1 (Feistel Network)	Mapeo bilingüe determinista de 15 caracteres	NIST SP 800-38G
Firmas Digitales	ML-DSA-65 / ML-DSA-85	Firmas basadas en celosías (CRYSTALS-Dilithium)	NIST FIPS 204
Intercambio Claves	ML-KEM-768	Encapsulamiento de claves TLS híbrido (Kyber)	NIST FIPS 203
Privacidad / ZKP	SD-JWT	Divulgación selectiva con hashes SHA-256	IETF SD-JWT Specification

Módulo	Estándar / Algoritmo	Implementación en OmniID	Cumplimiento
independientes			
Biometría Segura	Fuzzy Commitment Scheme	Criptosistema de paridad XOR BCH para vectores de minucias	Cero almacenamiento de biometría cruda
Cumplimiento GDPR	Arquitectura de 5 Pilares	AES-256-GCM, TLS 1.3, TOTP/WebAuthn, Audit Log inmutable	GDPR Art. 15, 16, 17, 20, 32



Estructura del Repositorio

El repositorio se divide en dos componentes principales: la interfaz de simulación web y el servidor API real:

```
|— index.html          # Interfaz gráfica del Simulador Web
|— styles.css          # Estilos en CSS nativo (Glassmorphism, 3D card)
|— app.js              # Lógica del cliente de simulación local
|— README.md           # Documentación general de GitHub
|— omniid_handover.md  # Guía de traspaso y continuidad técnica
└─ backend/            # Servidor API REST (Criptografía Real)
    |— package.json     # Gestión de dependencias de Node.js
    |— tsconfig.json    # Configuración del compilador TypeScript
    └─ src/
        |— server.ts     # Servidor Express, CORS y middlewares
        |— crypto/
        |   |— fpe.ts     # Generador de IDs FPE (Hull-Dobell LCG)
        |   └─ pqc.ts     # Firmas ML-DSA, Fuzzy BCH y SD-JWT
        |— routes/
        |   └─ identity.ts # API endpoints de enrolamiento y verificación ZKP
        └─ test_flow.ts   # Script de pruebas automatizadas
```



Componentes Implementados

1. Interfaz de Simulación (Frontend)

El cliente frontend simula un entorno completo en una sola pantalla:

- **Enrolamiento Biométrico:** Captura simulada de minucias dactilares y generación del ID de 15 caracteres.
- **Cédula Física 3D:** Tarjeta interactiva con efectos de inclinación tridimensional que muestra los niveles físicos (OVI, Kinegram, MRZ y chip NFC).
- **Mobile Wallet:** Simulador de smartphone que incluye bloqueo FaceID y generación de códigos **QR Dinámicos** válidos solo por 15 segundos (TTL).
- **Consola de Auditoría:** Terminal que detalla en tiempo real los logs del protocolo de verificación ZKP y las consultas DLT.

2. API Criptográfica (Backend)

El servidor en TypeScript procesa operaciones reales:

- **Format-Preserving Encryption:** Permutación matemática biyectiva que asocia contadores a identificadores estructurados sin riesgo de duplicados y sin necesidad de base de datos centralizada.
- **Fuzzy Commitment:** Permite atestar biometrías con ruido tolerable (por ejemplo, dedos húmedos o lecturas con variación angular) reconstruyendo el secreto dactilar de forma segura sin almacenar la huella cruda.
- **SD-JWT:** Firma y empaqueta credenciales atestadas, permitiendo al usuario decidir qué atributos (nombre, edad, ubicación) divulgar a terceros.

3. Cumplimiento GDPR (Backend Securizado)

Se ha integrado una arquitectura de 5 pilares para el estricto cumplimiento del Reglamento General de Protección de Datos (GDPR):

- **Seguridad (Art. 32):** Cifrado AES-256-GCM para datos en reposo, TLS 1.3 en tránsito, Rate Limiting, prevención de pérdida de datos (DLP), y detección automática de brechas (Art. 33).
- **Autenticación y Control de Acceso:** Autenticación Multifactor (MFA) soportando TOTP y WebAuthn/FIDO2, con gestión de sesiones seguras JWT y Control de Acceso Basado en Roles (RBAC).
- **Consentimiento y Auditoría (Art. 7, Art. 30):** Gestión granular de consentimientos por propósito, política de cookies, y un registro de auditoría inmutable (Audit Log) enlazado criptográficamente por hashes (SHA-256).

- **Derechos del Usuario (Art. 15, 16, 17, 20):** APIs automatizadas para el acceso a la información, exportación portable de datos (Portabilidad), rectificación, y eliminación de datos (Derecho al Olvido) soportado por un periodo de gracia.



Guía de Instalación y Ejecución

Requisitos Previos

- [Node.js](#) v20.0.0 o superior instalado.
- Navegador moderno con soporte para CSS Grid y 3D transforms.

Paso 1: Configurar el Servidor

Navegue al directorio del servidor e instale las dependencias:

```
cd backend  
npm install
```

Paso 2: Ejecutar los Tests de Integración

Antes de iniciar, valide que todos los algoritmos criptográficos funcionen en su entorno ejecutando el test suite:

```
npm run build  
npx ts-node src/test_flow.ts
```

Debería ver el mensaje ***¡TODAS LAS PRUEBAS PASARON EXITOSAMENTE!*** en la terminal.

Paso 3: Levantar la API

Inicie el servidor en modo desarrollo:

```
npm run dev
```

El servidor comenzará a escuchar en: <http://localhost:3000>.

Paso 4: Ejecutar el Frontend

Simplemente haga doble clic sobre el archivo [index.html] (file:///c:/Users/Joaquin%20Obando/Desktop/id%20global/index.html) de la raíz del proyecto para abrirlo directamente en el navegador, o sírvase de un servidor web local:

```
python -m http.server 8000
```

Y abra <http://localhost:8000>.



Referencia de la API (Endpoints)

1. Estado de la Red

- **Ruta:** [GET /api/v1/identities/status](#)
- **Descripción:** Devuelve la clave pública PQC maestra del emisor y el estado de la red DLT.
- **Respuesta Exitosa (200 OK):**

```
{  
  "success": true,  
  "network": "Hyperledger Indy",  
  "consensus": "RBFT Active",  
  "issuerPQCKey": "z6MkmL_ML-DSA-65_PK_..."  
}
```

2. Emisión de Cédulas Digitales (Enrolamiento)

- **Ruta:** `POST /api/v1/identities/issue`
- **Cuerpo (JSON):**

```
{
  "fullName": "Joaquin Obando",
  "dob": "2000-05-15",
  "region": "Managua, Nicaragua",
  "biometricTemplate": [142, 99, 44, 212, 85, 12, 190, 77]
}
```

- **Respuesta Exitosa (201 Created):** Devuelve el DID Document, el token en formato SD-JWT, el compromiso Fuzzy de biometría y las llaves privadas del dispositivo del Holder generadas dentro del Secure Enclave.

3. Verificación de Identidad (Auditoría ZKP)

- **Ruta:** `POST /api/v1/identities/verify`
- **Cuerpo (JSON):**

```
{
  "sdjwt": "header.payload.signature",
  "releasedDisclosures": ["WyJkdzdZ...5Jd"],
  "rule": "age",
  "verificationChallenge": "nonce_ejemplo",
  "holderSignature": "Sig_ML-DSA-65_..."
}
```

- **Respuesta Exitosa (200 OK):**

```
{
  "success": true,
  "approved": true,
  "disclosedClaims": {},
  "logs": [
    "Resolución de DID y atestación de validez en Ledger DLT Indy exitosa.",
    "Firma digital ML-DSA-85 del Emisor Nacional validada correctamente.",
    "ZKP validado: Edad (26 años) >= 18."
  ]
}
```

4. Endpoints de Cumplimiento GDPR

Adicionalmente, el sistema expone los siguientes módulos de API para garantizar la privacidad y los derechos del usuario:

- `GET /api/v1/consent/status` — Gestión y visualización de consentimientos otorgados.
- `GET /api/v1/audit/my-activity` — Transparencia de accesos sobre los datos del usuario.
- `POST /api/v1/gdpr/export` — Generación de paquete de datos portables (JSON/TXT).
- `POST /api/v1/gdpr/erase` — Ejecución del derecho al olvido con confirmación y periodo de gracia.

Plan de Continuidad y Próximas Fases

1. **Conexión Cliente-Servidor:** Migrar `app.js` del simulador cliente para consumir la API de Node.js a través de peticiones `fetch()`.
2. **Ledger Distribuido:** Integrar el SDK `@hyperledger/indy-vdr` para persistir los metadatos de DIDs en un nodo validador de prueba.
3. **Billetera Nativa:** Crear un módulo nativo en React Native que implemente las llamadas de firma ML-DSA directamente en la partición del microprocesador de seguridad móvil (**Secure Enclave** / **StrongBox**).



Licencia

Este proyecto está licenciado bajo la Licencia GNU Affero General Public License v3.0 (AGPL-3.0). Consulte el archivo [LICENSE] (file:///c:/Users/Joaquin%20Obando/Desktop/id%20global/LICENSE) para más detalles.